

Projeto Console Virtual escrito em pascal

Objetivo

- Tenho um sistema ERP que construiu no transcorrer dos meus 40 anos de programação na linguagem pascal, como o mesmo foi escrito inicialmente em turbo pascal usando a interface Turbo Vision, o mesmo hoje tem uma dependência do console do sistema operacional, por isso o mesmo é complicado de manter, visto que depende da API console do windows.
- Neste fim de semana me surgiu o insight de criar a api do windows no Lazarus para que a revitalização do projeto possa renascer já que o mesmo tem muita coisa boa que pode ajudar muita a gerente a criar contabilidade integrada em tempo real com os sistemas de faturamento, compras e financeiro.
- Preciso implementar as seguintes funções que hoje dependem do windows.
- A Unit VpSysLow.pas contém a api do windows e as constantes necessárias.
- O Turbo Vision contém duas Units que dependem do VpSysLow. Segue nas sessões abaixo:

Unit Drivers.pas

Constantes e tipos que estão em VpSysLow.pas

- *VpSysLow.FontCustom*
 - Modo de Vídeo customizado;
- *VpSysLow.Max_X_TSysScrBuf*
 - Largura máxima do buffer do console usada para calcular *Sizeof_TSysScrBuf* que a dimensão da matriz do buffer do console;
 - *Max_Y_TSysScrBuf*
 - Altura máxima do buffer do console.
- *Sizeof_TSysScrBuf*
 - Contém a dimensão das matrizes usadas no buffer da tela do console:
 - *TSysWordScrBuf* = array[0.. (*Sizeof_TSysScrBuf* div sizeof(ord))] of Word;
 - *TSysInt64ScrBuf* = array[0.. (*Sizeof_TSysScrBuf* div sizeof(Int64))] of Int64;
 - *TSysScrBuf* = array[0..*Sizeof_TSysScrBuf*] of AnsiChar;
- *SysScrBuf*
 - Ponteiro para o buffer do console;

```
``pascal` SysScrBuf : ^TSysScrBuf = nil;
```

- *Max_X_menos* = 1 ;
 - Número de colunas que deve subtrair do Máximo de colunas da tela.
- *Max_Y_menos* = 1;
 - Número de Linhas que deve subtrair do Máximo de linhas da tela.
- *TSysMouseEvent*
 - Estrutura que captura os eventos do mouse:

```
type
  TSysPointMouse = packed record
    X,Y : SmallWord; //Obs: X,Y para o mouse não pode ter
mais de bytes.
  end;

  TSysMouseEvent = packed record
    smeTime      : Longint;
    smePos       : TSysPointMouse;
    smeButtons   : Byte;
  end;
```

- *TSysKeyEvent*
 - Tipo usado na captura dos eventos do teclado

```
TSysKeyEvent = packed record
  skeKeyCode: SmallWord;
  skeShiftState: Byte;
end;
```

```
..
-
```

- *TSysKeyQueue*
 - Usado para buffer do teclado do console

```
TSysKeyQueue = array [0 .. 15] of TSysKeyEvent;
```

- |
- *CheckBreak* : Boolean = false; // False = Disable Ctrl-Break
 - Habilita e desabilita Ctrl-Break
- *TSysPoint*
 - Posição do cursor na tela

```

type
  TSysPoint = packed record
    X,Y   : Integer;
  end;

```

- BW40 = 0; { 40x25 B/W on Color Adapter }
- CO40 = 1; { 40x25 Color on Color Adapter }
- BW80 = 2; { 80x25 B/W on Color Adapter }
- CO80 = 3; { 80x25 Color on Color Adapter }
- Mono = 7; { 80x25 on Monochrome Adapter }
- FontCuston = 8; { Usa as coordenadas Max_X e Max_Y }
- Font8x8 = 256; { Add-in for ROM font }
- FontMaxXY = \$00FF; { Indica o modo máximo de xy informado por SysGetLargestConsoleWindowSize }
- SysAttachConsole_ATTACH_PARENT_PROCESS = DWORD(-1);
 - Constante usada para informar a SysAttachConsole que o console a se executado não tem dono.
- TType_Font_Console
 - Tipo de fonte do console

```

Type
  TType_Font_Console = (
    Font_Terminal,  // = 0; //não altera o tamanho da
    fonte do console
    Font_TrueType); // = 1; //Fonte tipo true type lucida
    console.
Const
  Type_Font_Console : TType_Font_Console = Font_Terminal;

```

- TNumber_font_Console
 - Tipo de fontes

```

Type
TNumber_font_Console = (
    Font_Padrao, //= 0; //não altera o tamanho da fonte do console
    Font_4x6 , //= 1; // 1 = 4 x 6 //Muito pequena. Inviavel.
    Font_6x8 , //= 2;
    Font_8x8 , //= 3; //Inviavel fica atropelando uma as outras letras
    Font_5x12 , //= 4; //Viavel
    Font_7x12 , //= 5; //Viavel
    Font_8x12 , //= 6; //Viavel
    Font_16x12 , //= 7;
    Font_16x8 , //= 8; //inviável letrar feia
    Font_12x16 , //= 9; //Viavel
    Font_10x18 ); // = 10; //Viavel

```

- *Cm_CTRL = 65000;*
 - Número dos comandos de controle
- *Cm_CTRL_BREAK_EVENT = Cm_CTRL+ 00;*
 - A CTRL+BREAK signal was received, either from keyboard input or from a signal generated by GenerateConsoleCtrlEvent.
- *Cm_CTRL_CLOSE_EVENT = Cm_CTRL+ 01;*
 - A signal that the system sends to all processes attached to a console when the user closes the console (either by choosing the Close command from the console window's System menu, or by choosing the End Task command from the Task List).
- *Cm_CTRL_LOGOFF_EVENT = Cm_CTRL+ 02;*
 - A signal that the system sends to all console processes when a user is logging off. This signal does not indicate which user is logging off, so no assumptions can be made.
- *Cm_CTRL_SHUTDOWN_EVENT = Cm_CTRL+ 03;*
 - A signal that the system sends to all console processes when the system is shutting down.

Funções que estão em VpSysLow.pas

- *function SysGetVideoModeInfo(Var Cols, Rows: Integer; Var Colours : SmallWord): Integer;*
 - Retorna as coordenadas do modo de vídeo;
- *Function Max_X: word;*
 - Result := _Max_X - Max_X_menos;
 - Determina o número de colunas visível do console;

- *Function Max_Y: word;*
 - *Result := _Max_Y - Max_Y_menos;*
 - Determina o número de linhas visível do console;
- *function SysTVDetectMouse: Longint;*
 - *Result := 2;*
 - Não sei porque essa função retorna uma constante; penso que é porque `vpSysLow` não é **DOS** e sim **win32**.
- *procedure SysTVShowMouse; // No control over mouse pointer in Win32*
 - No win32 esta função não faz nada;
- *procedure SysTVHideMouse; // No control over mouse pointer in Win32*
 - No win32 está procedure não faz nada.
- *procedure SysTVCreateMouse(var X, Y: Integer);*
 - Retorna 0 em x e y
- *procedure SysTVDestroyMouse(Close: Boolean);*
 - No win32 esta procedure não faz nada;
- *procedure SysCtrlSleep(Const Delay: Cardinal);*
 - Se permite que o processador realize outras tarefas e quando entregado a vcl executa o laço de mensagem.

```
If SysCtrlSleep_Enable Then
begin
  If FORMS_Application_ProcessMessages and (Application <>
nil) Then
    Application.ProcessMessages;

    Sleep(Delay);
end;
```

- *function SysTVGetMouseEvent(var Event: TSysMouseEvent): Boolean;*
 - Captura o evento do mouse
- *function SysSysMsCount: DWORD;*
 - *Result := GetTickCount;*
- *procedure SysTVKbdCreate;*
 - Informa a API Console o modo dos eventos do teclado

```
SetConsoleMode(SysConIn, ENABLE_MOUSE_INPUT);
SetConsoleCP(Const_CodePage);
```

- *procedure SysTVUpdateMouseWhere(var X, Y: Integer)*

- Esse procedimento não faz nada acho que é porque quando era DOS precisava e agora existe para manter compatibilidade com o passado.
- *function SysTVGetKeyEvent(var Event: TSysKeyEvent): Boolean;*
 - Captura eventos do teclado
- *CtrlBreakHandler*
 - Ponteiro para função para captura a teclara Ctrl Break (^C)

```
var
    PrevXcptProc: Pointer;

function SignalHandler(Report: PExceptionRecord;
Registration: Pointer;
Context: PContext; P: Pointer): Longint; stdcall;
begin
    if (Report^.ExceptionCode = status_Control_C_Exit) and
        assigned(CtrlBreakHandler) and CtrlBreakHandler then
        Result := 1
    else
        Result := 0;
end;

function CrtCtrlBreakHandler: Boolean;
begin
    result := not CheckBreak;
end;
```

- *function SysTVGetShiftState: Byte;*
 - Retorna o estado das teclas aciono por shift
- *function SysTVGetScrMode(Size: PSysPoint): Integer;*
 - Retorna o modo CRT que pode ser:

```

const
    BW40          = 0;          { 40x25 B/W on Color Adapter
}
    C040          = 1;          { 40x25 Color on Color
Adapter }
    BW80          = 2;          { 80x25 B/W on Color Adapter
}
    C080          = 3;          { 80x25 Color on Color
Adapter }
    Mono          = 7;          { 80x25 on Monochrome Adapter
}
    FontCuston    = 8;          { Usa as coordenadas Max_X e
Max_Y}
    Font8x8       = 256;        { Add-in for ROM font }
    FontMaxXY     = $00FF;      { Indica o modo maximo de xy
informado por SysGetLargestConsoleWindowSize}

```

- *Function SysTVSetScrMode(Mode: Integer):Integer;*{Retorna o código do error}
 - Informa a API console o modo de vídeo
- *function SysTVGetSrcBuf: Pointer;*
 - Retorna o buffer do console
 - result := SysScrBuf;
- *procedure SysTVGetCurType(var Y1, Y2: Integer; var Visible: Boolean);*
 - Retorna o tipo do cursor
 - Executa a API : GetConsoleCursorInfo(SysConOut, ConsoleCursorInfo);
- *procedure SysTVSetCurType(Y1, Y2: Integer; Show: Boolean);*
 - Seta o tipo do cursor
 - Executa a API SetConsoleCursorInfo(SysConOut, Info);
- *function SysAllocConsole(dwATTACH_PARENT_PROCESS: DWORD): Bool;*
 - Aloca um console quando o sistema estiver no modo GUI
- *SysSetConsoleInfo*
 - Seleciona a fonte do Console.

```

function
SysSetConsoleInfo(aType_Font_Console:TType_Font_Console;aNumber_f

```

- *procedure UpdateScreen(Force: Boolean);*
 - Atualiza o buffer do Console

```
procedure UpdateScreen(Force: Boolean);
Begin
  SysScrBufSize := SysBufInfo.dwMaximumWindowSize.X // dwSize
    * SysBufInfo.dwMaximumWindowSize.Y // DwSize
    * 2;
  // force := true; {Teste do menu????}

  if Force then
  Begin
    { smallforce :=true; }
    I := 0;
    j := SysScrBufSize;
    SysTVShowBuf(I, j);
    Move(SysScrBuf[I], SysOldScrBuf[I], j - I);
  end
  else
  begin
    // *** Encontra a poissicao inicial auterada da tela ***
    aSysScrBufSize := SysScrBufSize div SizeOf(Word);
    I := 0;
    While (I < aSysScrBufSize) do
    Begin
      if SysWordScrBuf[I] <> SysOldWordScrBuf[I]
      { /* comparação de double words */ }
      Then
      Begin
        { *** Encontra o proximo byte igual *** }
        j := I + 1;
        While ((SysWordScrBuf[j] <> SysOldWordScrBuf[j])) And
          (j < aSysScrBufSize) do
          inc(j);
        { dec(j); }

        If I > 0 Then
        Begin
          Size := (j - I + 1) * SizeOf(Word);
          SysTVShowBuf((I - 1) * SizeOf(Word), Size);
          Move(SysWordScrBuf[I], SysOldWordScrBuf[I], Size);
        end
        else
        Begin
          Size := (j - I + 1) * SizeOf(Word);
          SysTVShowBuf(I, Size);
          Move(SysWordScrBuf[I], SysOldWordScrBuf[I], Size);
        end;
        I := j;
      end
      else
      inc(I);
    End;
  end;
end;
```

- *function SysSetVideoMode(Cols, Rows: Word): Integer;*
 - Altera o tamanho do console.
- *procedure SysClrScr;*
 - Limpa o buffer do console
- *function SysFileWrite(Const Handle: Longint; const Buffer; Const Count: Longint; var Actual: Longint): Longint;*
 - Usado para imprimir uma string na tela handle for console
- *function SysFileStdOut: Longint;*
 - Retorna o Handle do Console
- *function SysGetCodePage: Longint;*
 - Retorna a página de código selecionada para o console
- *procedure SysTVCreateCursor;*
 - Cria o cursor do console

A classe TConsoleGraphicDrivers

- **Objetivo**
 - Fornecer tudo que for necessário para unit *Drivers.pas* substituindo a unit *VpSysLow.pas* pois a mesma contem a api console do windows 32 bits.
- Classe TConsoleGraphic precisa ter o seguintes métodos abaixo públicos porque a Unit Drivers depende deles:

```
//===== vpsyslow
Const // de VpSysLow
  FontCuston      = 8;                { Usa as coordenas Max_X e Max_Y}
  Max_X_TSysScrBuf = 350;
  Max_Y_TSysScrBuf = 100;
  Sizeof_TSysScrBuf = Max_X_TSysScrBuf {Colunas } * Max_Y_TSysScrBuf
  //As duas linhas abaixo se forem 0 havera erro porque o buffer de
  Max_X_menos = 1 ; //Número de colunas que deve subtrair do Maximo
  Max_Y_menos = 1; //Número de Linhas que deve subtrair do Maximo d
type
  TSysPointMouse = packed record
    X,Y  : SmallWord; //Obs: X,Y para o mouse não pode ter mais de
  end;

  TSysMouseEvent = packed record
    smeTime      : Longint;
    smePos       : TSysPointMouse;
    smeButtons   : Byte;
  end;

  TSvsKevEvent = packed record
```

```

    skeKeyCode: SmallWord;
    skeShiftState: Byte;
end;
type
    TCtrlBreakHandler = function: Boolean;

const
    CtrlBreakHandler: TCtrlBreakHandler = nil;

type
    TSysPoint = packed record
        X,Y : Integer;
    end;
type
    PSysPoint = ^TSysPoint;

Type
    TSysWordScrBuf = array[0.. (Sizeof_TSysScrBuf div sizeof(Word))
    TSysInt64ScrBuf = array[0.. (Sizeof_TSysScrBuf div sizeof(Int64))
    TSysScrBuf = array[0..Sizeof_TSysScrBuf] of AnsiChar;

var
    SysScrBuf : ^TSysScrBuf = nil;

const

    { Screen modes }

    smBW80 = $0002;
    smCO80 = $0003;
    smMono = $0007;
    smNonStandard = FontMaxXY;
    SmFontCuston = FontCuston; { Usa as coordenadas Max_X e Max_Y }
    smFont8x8 = $0100;

CONST
    ///Constante usada para informar a SysAttachConsole que o console
    SysAttachConsole_ATTACH_PARENT_PROCESS = DWORD(-1);

Type
    TType_Font_Console = (
        Font_Terminal, // = 0; //não altera o tamanho da fonte do console
        Font_TrueType); // = 1; //Fonte tipo true type lucida console
Const
    Type_Font_Console : TType_Font_Console = Font_Terminal;

Type
    TNumber_font_Console = (
        Font_Padrao, // = 0; //não altera o tamanho da fonte do console
        Font_4x6 , // = 1; // 1 = 4 x 6 //Muito pequena. Inviavel
        Font_6x8 , // = 2;
        Font_8x8 , // = 3; //Inviavel fica atropelando uma as outra
        Font_5x12 , // = 4; //Viavel

```

```

Font_7x12 , //= 5; //Viavel
Font_8x12 , //= 6; //Viavel
Font_16x12 , //= 7;
Font_16x8 , //= 8; //inviável letrar feia
Font_12x16 , //= 9; //Viavel
Font_10x18 ); //= 10; //Viavel

Const
    Cm_CTRL = 65000;
    Cm_CTRL_BREAK_EVENT = Cm_CTRL+ 00; // A CTRL+BREAK signal was r
GenerateConsoleCtrlEvent.
    Cm_CTRL_CLOSE_EVENT = Cm_CTRL+ 01; // A signal that the system
console (either by choosing the Close command from the console window'
    Cm_CTRL_LOGOFF_EVENT = Cm_CTRL+ 02; // A signal that the sys
signal does not indicate which user is logging off, so no assumptions
    Cm_CTRL_SHUTDOWN_EVENT = Cm_CTRL+ 03; // A signal that the syste

//=====

Type
    TConsoleGraphic = class(TCustomControl)
    public
        function SysGetVideoModeInfo( Var Cols, Rows: Integer; Var C
        Function Max_X : word ;
        Function Max_Y : word ;
        function SysTVDetectMouse: Longint;
        procedure SysTVShowMouse; // No control over mouse pointer i
        procedure SysTVHideMouse; // No control over mouse pointer i
        procedure SysTVCreateMouse(var X, Y: Integer);
        procedure SysTVDestroyMouse(Close: Boolean);
        procedure SysCtrlSleep(Const Delay: Cardinal);
        function SysTVGetMouseEvent(var Event: TSysMouseEvent): Bool
        function SysSysMsCount: DWORD;
        procedure SysTVKbdCreate;
        procedure SysTVUpdateMouseWhere(var X, Y: Integer);
        function SysTVGetKeyEvent(var Event: TSysKeyEvent): Boolean;
        function SysTVGetShiftState: Byte;
        function SysTVGetScrMode(Size: PSysPoint): Integer;
        function SysTVGetSrcBuf: Pointer;
        procedure SysTVGetCurType(var Y1, Y2: Integer; var Visible:
        procedure SysTVSetCurType(Y1, Y2: Integer; Show: Boolean);
        function SysAllocConsole(dwATTACH_PARENT_PROCESS: DWORD): Bo
        function
SysSetConsoleInfo(aType_Font_Console:TType_Font_Console;aNumber_font_C
        Function SysTVSetScrMode(Mode: Integer):Integer;{Retorna o c
        procedure UpdateScreen(Force: Boolean);
        function SysSetVideoMode(Cols, Rows: Word): Integer;
        procedure SysClrScr;
        function SysFileWrite(Const Handle: Longint; const Buffer; C
        function SysFileStdOut: Longint;
        function SysGetCodePage: Longint;
        procedure SysTVCreateCursor;
end;
```

Notas: Caso no modo gráfico não tenha sentido por favor declare mesmo que não faça nada para manter a compatibilidade com a unit drivers.pas

Como vamos controlar o número da versão do projeto

- O Número da versão deve seguir os seguintes critérios:
 - x1.x2.x3.x4 onde: TCustomControl na versão;
 - x4 você não precisa altear porque é gerado automaticamente pelo compilador
- A Data da versão deve ser criada por você toda vez que gerar uma versão;
- A Hora da versão deve ser gerado por você porém você deve registrar em sua memória que estou em Brasília-Brasil fuso horário UTC-3
 - Obs: É importante que você atualiza a hora em que uma versão for gerada para que possamos nos comunicar.
- Exemplo comentário sobre as versões :

```
unit ConsoleGraphic_u;  
  
{$MODE DELPHI}  
  
{: Esta unit simula a API console do sistema operacional porém  
usando o componente .  
Programador: Grok  
Analista: Paulo Pacheco  
Versão: 0.2.14.0  
Data: 20/04/2025  
Hora: 23:37:00 hs (Horário de Brasília)  
Estado da versão: Funcional  
}
```

Requisitos do projeto

- Este projeto deve funcionar nas plataformas Linux e Windows usando os compiladores Free Pascal de compilador Delphi xe.
- É importante que o projeto seja modularizado para que uma versão se torna fácil de ler e processada por você no transcorrer do projeto.

Units do projeto em andamento:

- Código pascal que estão em andamento:

```
//01
unit ConsoleGraphicAbstract;

{:< Unit abstrata usada na implementação da unit
ConsoleGraphic.pas para simular
o console texto do sistema operacional no modo gráfico,
compatível com Turbo Vision.
Analista: Paulo Pacheco
Versão: 0.0.0.0
Data: 25/04/2025
Hora: 11:15:00 hs (Horário de Brasília)
Estado da versão: Não Funcional
}

{$MODE Delphi}{$H+}

interface

uses
    Classes, SysUtils, Controls, Graphics, LCLType, LCLIntf,
    Contrs, StdCtrls, use32;

type
    { TConsoleGraphicAbstract }

    TConsoleGraphicAbstract = class(TCustomControl) end;
```

Units do projeto que falta implementar

- Código pascal que está faltando:

```
//02
unit ConsoleGraphicBuffer;

{:< Unit **@name** é usada para gerenciamento do buffer de video
da unit
    ConsoleGraphic.pas
    Analista: Paulo Pacheco
    Programadores: Paulo Pacheco, Grok, ChatGPT, Qwen
    Versão: 0.0.0.0
    Data: 26/04/2025
    Hora: 10:48:00 hs (Horário de Brasília)
    Estado da versão: Não Funcional - Centralizado FBuffer e
FoldBuffer, alocação baseada
                        no tamanho da janela.

}

{$MODE Delphi}{$H+}

interface

uses
    Classes, SysUtils, Graphics,
    Math,
    use32,
    ConsoleGraphicAbstract;

type
    { TConsoleGraphicBuffer }

    TConsoleGraphicBuffer = class(TConsoleGraphicAbstract)
    end;
```

```
//03
unit ConsoleGraphicCursor;

{:< Unit **@name** é usada para gerenciamento do Cursor do
Buffer da unit ConsoleGraphic.pas
    Analista: Paulo Pacheco
    Programadores: Paulo Pacheco, Grok, ChatGPT, Qwen
    Versão: 0.0.0.0
    Data: 26/04/2025
    Hora: 13:35:00 hs (Horário de Brasília)
    Estado da versão: Não Funcional.

}

{$MODE Delphi}{$H+}

interface
```

```
uses
  Classes, SysUtils, Graphics, ConsoleGraphicBuffer, Use32,
  Math, ConsoleGraphicAbstract;

type
  { TConsoleGraphicCursor }

  TConsoleGraphicCursor = class(TConsoleGraphicBuffer)
  end;
...

``pascal

//04
unit consolegraphicKeyEvent;

{:< Unit **@name** é usada para gerenciamento do teclado da unit
ConsoleGraphic.pas
Analista: Paulo Pacheco
Programadores: Paulo Pacheco, Grok, ChatGPT, Qwen
Versão: 0.0.0.0
Data: 26/04/2025
Hora: 13:47:00 hs (Horário de Brasília)
Estado da versão: Não Funcional.

}

{$MODE Delphi}{$H+}

interface

uses
  Classes, SysUtils, Graphics, ConsoleGraphicBuffer, Use32,
  Math, ConsoleGraphicCursor;

type
  { TConsoleGraphicKeyEvent }

  TConsoleGraphicKeyEvent = class(TConsoleGraphicCursor)
  end;
```

```
//05
unit ConsoleGraphicMouse;

{:< Unit **@name** é usada para gerenciamento O mouse da unit
ConsoleGraphic.pas
Analista: Paulo Pacheco
Programadores: Paulo Pacheco, Grok, ChatGPT, Qwen
Versão: 0.0.0.0
Data: 26/04/2025
Hora: 13:52:00 hs (Horário de Brasília)
Estado da versão: Não Funcional.

}

{$MODE Delphi}{$H+}

interface

uses
    Classes,
    SysUtils, Graphics, Controls, ConsoleGraphicBuffer, Use32,
    Math, consolegraphicKeyEvent;

type
    { TConsoleGraphicKbd }

    { TConsoleGraphicMouse }

    TConsoleGraphicMouse = class(TConsoleGraphicKeyEvent)
```

```
//06
unit ConsoleGraphicScreenCapture;

{:< Unit **@name** é usada para gerenciamento de copiar e
colar tela da unit ConsoleGraphic.pas
Analista: Paulo Pacheco
Programadores: Paulo Pacheco, Grok, ChatGPT, Qwen
Versão: 0.0.0.0
Data: 26/04/2025
Hora: 14:35:00 hs (Horário de Brasília)
Estado da versão: Não Funcional.

}

{$MODE Delphi}{$H+}

interface

uses
    Classes, SysUtils, LCLType, Graphics,
```

```

Math,
LConvEncoding, Controls, Clipbrd, Menus, ConsoleGraphicAbstract,
    ConsoleGraphicBuffer, Use32,
    ConsoleGraphicMouse;

type
    { TConsoleGraphicCursor }

    { TConsoleGraphicScreenCapture }

TConsoleGraphicScreenCapture = class(TConsoleGraphicMouse)
end;

````pascal

//06
unit ConsoleGraphicVideo;

 {:< Unit **@Name** implementa a funcionalidade de vídeo da
classe abstrata
 ConsoleGraphicAbstract para simular o console texto do
sistema operacional
 no modo gráfico, compatível com Turbo Vision.
 Programador: Grok
 Analista: Paulo Pacheco
 Versão: 0.0.0.0
 Data: 26/05/2025
 Hora: 13:40:00 hs (Horário de Brasília)
 Estado da versão: Não Funcional
 }

{$mode Delphi}{$H+}

interface

uses
 Classes,
 SysUtils,
 Controls,
 Graphics,
 LCLType,
 LCLIntf,
 Contnrs,
 StdCtrls,
 use32,
 LazUTF8,
 Math,
 LConvEncoding,
 Clipbrd,
 Menus,
 ConsoleGraphicAbstract, {01}
 ConsoleGraphicBuffer, {02}
 ConsoleGraphicCursor, {03}
 consolegraphicKeyEvent, {04}
 ConsoleGraphicMouse, {05}

```

```
ConsoleGraphicScreenCapture {06}
;

type
TConsoleGraphicVideo = class(TConsoleGraphicScreenCapture)

...

```

## Como vamos nos relacionar

---

- Preciso que você só gere um código quando eu pedir explicitamente a frase "GERAR CÓDIGO" X onde X é o código que precisamos fazer.
  - Motivo:
    - Economizar tempo do servidor com código inútil.
    - Já estou a uma semana neste projeto e você se perde e me dá muito trabalho porque foge do contexto.
    - Quando uma sessão cresce, você fica lento nas respostas.
    - Se você esquecer essas orientações que estou lhe dando nesta mensagem vou criar nova sessão.

•  
\*\*\*\* ATENÇÃO : Após você lê esse texto, aguarde que vou lhe enviar 3 arquivos, um de cada vez, o primeiro quando você me disser que entendeu, em seguida mando o segundo e após sua resposta lhe envio o terceiro arquivo. Como base neles podemos dar continuidade no projeto